Weighted Graphs

◆ PROMPT 1: What is a weighted graph?

✅ Answer:

A **weighted graph** is a graph in which **each edge has a numerical value** called its **weight**.

- The weight often represents:
  - distance
  - cost
  - time
  - capacity
  - price

Formally:

- For an edge `e = (u, v)`
- The weight is written as
  w(e) or w(u, v)

◆ PROMPT 2: How is a weighted graph different from an unweighted graph?

✅ Answer:

Unweighted Graph	Weighted Graph
All edges equal	Edges have costs
Path length = number of edges	Path length = sum of weights
BFS works for shortest path	Dijkstra / Bellman-Ford needed
Simple structure	More realistic modeling

◆ PROMPT 3: Are weighted graphs directed or undirected?

✅ Answer:

Weighted graphs can be:

- **Undirected weighted**
- **Directed weighted**

Example:

- Undirected → road distance between cities
- Directed → one-way flights, network delays

◆ PROMPT 4: How do we represent weights mathematically?

✅ Answer:

For an edge between vertices `u` and `v`:

$$w(u,v) = weight\ of\ edge\ (u,v)$$

For a path $P = v_0, v_1, \ldots, v_k$

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

◆ PROMPT 5: What is a weighted path?

✅ Answer:

A **weighted path** is a sequence of edges where the **path cost** equals the **sum of edge weights**.

Example:

`JFK → ORD → DFW → LAX`

$$w = 740 + 802 + 1235 = 2777$$

◆ PROMPT 6: What is a minimum-weight (shortest) path?

✅ Answer:

A **minimum-weight path** is a path between two vertices whose **total weight is smallest among all possible paths**.

⚠ It is **not necessarily** the path with the fewest edges.

◆ PROMPT 7: Explain Figure 14.14 (Airport graph)

✅ Answer:

- **Vertices** → Airports (JFK, ORD, LAX, etc.)
- **Edges** → Flight routes
- **Weights** → Distance in miles

Shortest path from JFK to LAX:

```
JFK → ORD → DFW → LAX
```

Total weight:

```
740 + 802 + 1235 = 2777 miles
```

This is the **minimum-weight path**.

◆ PROMPT 8: Draw a weighted graph (ASCII diagram)

🖊 Example Weighted Graph

```
      740
  JFK ----- ORD
   |          |
   |1090      |802
   |          |
  MIA ----- DFW
      1235
        \
         \
          LAX
```

◆ PROMPT 9: How are weighted graphs stored in memory?

✅ Answer:

1️⃣ Adjacency List (most common)

```
JFK → (ORD,740), (MIA,1090)

ORD → (JFK,740), (DFW,802)
```

2️⃣ Adjacency Matrix

```
      JFK  ORD  DFW

JFK    0   740   ∞

ORD   740   0   802

DFW    ∞   802   0
```

◆ PROMPT 10: What problems require weighted graphs?

✅ Answer:

Problem	Algorithm
Shortest path	Dijkstra
Negative weights	Bellman-Ford
All-pairs shortest path	Floyd–Warshall
Minimum Spanning Tree	Prim, Kruskal
Network routing	Dijkstra
Road maps	Weighted graphs

◆ PROMPT 11: Can weights be negative?

✅ Answer:

Yes, but:

- **Dijkstra does NOT allow negative weights**

- **Bellman-Ford DOES**

Negative weights represent:

- profit
- energy gain
- credits

◆ PROMPT 12: What is the difference between shortest path and MST?

✅ Answer:

Shortest Path	Minimum Spanning Tree
Between two vertices	Spans all vertices
Minimizes path cost	Minimizes total edge cost
Depends on source	Independent of source
Uses Dijkstra	Uses Prim / Kruskal

◆ PROMPT 13: Key exam points (must remember)

✅ Answer:

- Weighted graph → edges have weights
- Path cost = sum of weights
- Minimum-weight path ≠ minimum edges
- Used in real-world networks
- Fundamental to MST and shortest paths

Defining Shortest Paths in a Weighted Graph

◆ PROMPT 1: What is a path in a weighted graph?

✅ Answer:

A **path** is a sequence of vertices connected by edges.

If

$$P = ((v_0,v_1),(v_1,v_2),\ldots,(v_{k-1},v_k))$$

then the path goes from v_0 to v_k using k edges.

◆ PROMPT 2: What is the length (weight) of a path?

✅ Answer:

The **length (or weight)** of a path is the **sum of the weights of its edges**.

Formally:

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

So:

- We **add**, not count
- Edge weights matter, not number of edges

◆ PROMPT 3: What is distance between two vertices?

✅ Answer:

The **distance** from vertex `u` to vertex `v`, written as **d(u,v)**, is:

The weight of the **minimum-weight path** from `u` to `v`

This path is called the **shortest path**.

◆ PROMPT 4: What if no path exists between u and v?

✅ Answer:

If **no path exists**, we define:

$$d(u,v) = \infty$$

This simply means **v is unreachable from u**.

◆ PROMPT 5: Why is distance sometimes undefined even when paths exist?

✅ Answer:

Distances become **undefined** when the graph contains a
🕳️ **negative-weight cycle**.

◆ PROMPT 6: What is a negative-weight cycle?

✅ Answer:

A **negative-weight cycle** is a cycle whose **total edge weight is negative**.

Example:

```
JFK → ORD → JFK

cost = -5 + -3 = -8
```

Each time you loop:

- Path cost becomes **smaller**
- You can loop **infinitely**
- Distance can go to $-\infty$

So a "shortest" path **does not exist**.

◆ PROMPT 7: Why do negative cycles break shortest paths?

✅ Answer:

Because you can:

- Reach a destination
- Loop the negative cycle any number of times
- Make the total cost **arbitrarily small**

Distances **cannot be** $-\infty$, so they are **undefined**.

◆ PROMPT 8: Real-world interpretation of negative cycles

✅ Answer:

In the example:

- Cities = vertices
- Costs = edge weights
- Someone **pays you to travel**

If:

- JFK → ORD pays you
- ORD → JFK pays you

You could loop forever and **earn unlimited money** 🤑
→ distances stop making sense.

◆ PROMPT 9: When are shortest paths guaranteed to exist?

✅ Answer:

Shortest paths are well-defined when:

- **All edge weights are non-negative**

$w(e) \geq 0$w(e)≥0

This guarantees:

- No negative cycles
- Distances are finite

◆ PROMPT 10: What problem are we trying to solve here?

✅ Answer:

This is the **single-source shortest path problem**:

Given a source vertex `s`, find the shortest path from `s` to **every other vertex**

◆ **PROMPT 11: How does BFS relate to shortest paths?**

✅ Answer:

If:

- All edges have weight **1**

Then:

- Shortest path = fewest edges
- **BFS solves it optimally**

For weighted graphs:

- BFS ❌ fails
- We need **greedy algorithms**

◆ PROMPT 12: What design pattern is used to solve this?

✅ Answer:

The **Greedy Method**

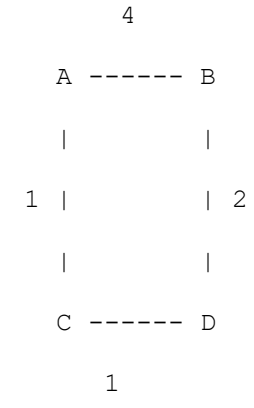
Idea:

- Start from source `s`
- Repeatedly select the **closest unvisited vertex**
- Lock in its shortest distance

This leads directly to **Dijkstra's algorithm**.

◆ PROMPT 13: Visual example (weighted shortest path)

🖊 Diagram



Paths from A to D:

- A → B → D = 4 + 2 = **6**
- A → C → D = 1 + 1 = **2** ✅ shortest

So:

$d(A,D) = 2$d(A,D)=2

◆ PROMPT 14: Key rules to remember (exam gold ⭐)

✅ Answer:

- Path weight = sum of edge weights
- Distance = minimum path weight
- No path → distance = ∞
- Negative cycles → distance undefined
- BFS works only if all weights = 1
- Dijkstra works if all weights ≥ 0

◆ PROMPT 15: What comes next after this section?

✅ Answer:

This section **sets up**:

- **Dijkstra's Algorithm**
- Greedy shortest-path strategy
- Priority queues

Dijkstra's Algorithm

◆ PROMPT 1: What problem does Dijkstra's algorithm solve?

✅ Answer:

Dijkstra's algorithm solves the **single-source shortest path problem**:

Given a weighted graph **G** with **nonnegative edge weights** and a source vertex **s**, compute the shortest distance from **s** to every other vertex reachable from **s**.

◆ PROMPT 2: Why is Dijkstra called a "weighted BFS"?

✅ Answer:

- **BFS** explores vertices by increasing number of edges
- **Dijkstra** explores vertices by increasing **total path weight**

So:

- BFS → unweighted graphs
- Dijkstra → weighted graphs (nonnegative weights)

◆ PROMPT 3: What is the "cloud" in Dijkstra's algorithm?

✅ Answer:

The **cloud C** is the set of vertices whose **shortest distance from s is finalized**.

Once a vertex enters the cloud:

- Its distance is **correct**
- It will **never change**

◆ PROMPT 4: What are the labels D[v]?

✅ Answer:

For each vertex v:

$$D[v] = best\ known\ distance\ from\ s\ to\ v$$

Meaning:

- An **estimate**
- Improves over time
- Becomes final when v enters the cloud

◆ PROMPT 5: How are the labels initialized?

✅ Answer:

- $D[s] = 0$
- $D[v] = \infty$ for all $v \neq s$
- Cloud $C = \varnothing$

◆ PROMPT 6: Why is a priority queue used?

✅ Answer:

At each step, we must select:

The vertex **outside the cloud** with the **smallest D[v]**

A **priority queue** does this efficiently.

◆ PROMPT 7: What happens in each iteration?

✅ Answer:

1. Remove the vertex `u` with smallest `D[u]` from the priority queue
2. Add `u` to the cloud
3. Relax all edges `(u,v)` where `v` is outside the cloud

◆ PROMPT 8: What is edge relaxation?

✅ Answer:

Edge relaxation checks whether going through `u` gives a shorter path to `v`.

Relaxation Rule:

```
if D[u] + w(u,v) < D[v]:
    D[v] = D[u] + w(u,v)
```

◆ PROMPT 9: Why is it called “relaxation”?

✅ Answer:

Because it:

- Takes a **tight** upper bound
- Relaxes it toward the **true shortest distance**

◆ PROMPT 10: Pseudocode meaning (core logic)

✅ Answer (plain English):

- Keep all vertices in a priority queue
- Repeatedly pick the closest vertex
- Lock its distance
- Improve neighbors’ distances if possible
- Stop when all reachable vertices are finalized

◆ PROMPT 11: When does the algorithm terminate?

✅ Answer:

- When the priority queue is empty
- Or when remaining vertices are unreachable (distance = ∞)

◆ PROMPT 12: Simple visual intuition (cloud growth)

```
Iteration 1:

s

Iteration 2:

s ----> u1

Iteration 3:

s ----> u1 ----> u2

Iteration 4:

s ----> u1 ----> u2 ----> u3
```

Vertices are added **in order of increasing distance**.

◆ PROMPT 13: Why does Dijkstra require nonnegative weights?

✅ Answer:

If edge weights were negative:

- A shorter path might appear **after** a vertex is finalized
- The greedy choice would fail

Nonnegative weights guarantee:

$D[u] \leq D[v] for\ all\ unvisited\ v$

◆ PROMPT 14: What does Proposition 14.23 state?

✅ Answer:

When a vertex `v` is pulled into the cloud, `D[v]` equals the true shortest distance `d(s,v)`.

◆ PROMPT 15: Why is this proposition important?

✅ Answer:

It proves:

- Dijkstra is **correct**
- Once a vertex enters the cloud, its distance is final

◆ PROMPT 16: Key idea behind the proof

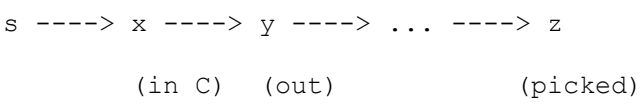
✅ Answer:

The proof uses **contradiction**:

- Assume the algorithm chooses a vertex with a wrong distance
- Show this violates the greedy choice
- Conclude such a vertex cannot exist

◆ PROMPT 17: Explanation of Figure 14.18 (core proof idea)

🖊 Diagram



Facts:

- `x` is already finalized
- `y` should have been chosen before `z`
- But `z` was chosen first → contradiction

◆ PROMPT 18: Why does the contradiction arise?

✅ Answer:

Because:

- Shortest paths have **nonnegative edges**
- Distance to `y` must be ≤ distance to `z`
- Priority queue should have picked `y` first

Thus:

$$D[z] \leq D[y] \leq d(s,z)$$

Contradiction!

◆ PROMPT 19: What guarantees correctness?

✅ Answer:

1. Greedy selection of minimum-distance vertex
2. No negative edge weights
3. Relaxation preserves shortest-path properties

◆ PROMPT 20: Final output of Dijkstra's algorithm

✅ Answer:

- Array `D[v]` → shortest distances
- (Optionally) parent pointers → shortest-path tree

◆ PROMPT 21: Time complexity (important for exams)

✅ Answer:

Depends on priority queue:

- Binary heap:
- $O((V + E)\, log\, V$
- Array-based PQ:
- $O(V^2)$

◆ PROMPT 22: When should you NOT use Dijkstra?

✅ Answer:

❌ If graph contains **negative-weight edges**

Use:

- **Bellman–Ford** instead

Programming Dijkstra's Algorithm in Python

◆ PROMPT 1: What does this Python function do?

✅ Answer:

`shortest_path_lengths(g, src)` computes the **shortest-path distance** from a **source vertex** **`src`** to **every reachable vertex** in a **weighted graph** **`g`** with **nonnegative edge weights**.

Output:

A dictionary called `cloud`:

```
cloud[v] = d(src, v)
```

◆ PROMPT 2: Why does the function return a dictionary instead of a list?

✅ Answer:

Because:

- Graph vertices may not be numbered `0…n-1`
- Dictionaries give **O(1) expected-time access**
- Works naturally with object-based graph implementations

◆ PROMPT 3: What are the main data structures used?

✅ Answer:

Variable	Purpose
`d`	Best known distance estimate `D[v]`
`cloud`	Finalized shortest distances
`pq`	Priority queue ordered by distance
`pqlocator`	Allows updating PQ keys efficiently

◆ PROMPT 4: Why use an *Adaptable* Heap Priority Queue?

✅ Answer:

Because Dijkstra needs to:

- **Decrease keys** when a shorter path is found
- Efficiently update an existing entry

☞ Standard heaps **cannot do this efficiently** without locators.

◆ PROMPT 5: What does this initialization step mean?

```
for v in g.vertices():
    if v is src:
        d[v] = 0
```

```
    else:
        d[v] = float('inf')
    pqlocator[v] = pq.add(d[v], v)
```

✅ Answer:

- Source starts with distance `0`
- All other vertices start with distance ∞
- All vertices are inserted into the priority queue

◆ PROMPT 6: Why use `float('inf')`?

✅ Answer:

It represents **positive infinity**, allowing:

- Numeric comparisons
- Safe relaxation checks
- Clean algorithm logic

◆ PROMPT 7: What does the priority queue contain?

✅ Answer:

Each PQ entry is:

```
(key = d[v], value = v)
```

Vertices are ordered by **current shortest distance estimate**.

◆ PROMPT 8: What does this loop represent?

```
while not pq.isempty():
```

✅ Answer:

Each iteration:

- Selects **one vertex**
- Finalizes its shortest-path distance
- Relaxes outgoing edges

This is the **core greedy loop**.

◆ PROMPT 9: What happens when we remove the minimum?

```
key, u = pq.removemin()
```

✅ Answer:

- `u` is the vertex with smallest distance estimate
- `key` is the correct shortest distance to `u`
- `u` enters the **cloud**

◆ PROMPT 10: Why is this distance correct?

✅ Answer:

Because:

- All edge weights are nonnegative
- Greedy choice ensures no shorter path exists
- Proven by Proposition 14.23

◆ PROMPT 11: What is the cloud?

```
cloud[u] = key
```

✅ Answer:

`cloud` stores **finalized distances**.

Once a vertex enters the cloud:

- Its distance is correct
- It will never change

◆ PROMPT 12: Why delete the locator?

```
del pqlocator[u]
```

✅ Answer:

Because:

- `u` is no longer in the priority queue
- We must not attempt to update it again

◆ PROMPT 13: What does this loop process?

```
for e in g.incident_edges(u):
```

✅ Answer:

It iterates over **all outgoing edges** `(u, v)`
(For undirected graphs, both directions)

◆ PROMPT 14: What does this line do?

```
v = e.opposite(u)
```

✅ Answer:

Given edge `(u, v)`, it returns the **other endpoint**.

◆ PROMPT 15: Why skip vertices already in the cloud?

```
if v not in cloud:
```

✅ Answer:

Vertices in the cloud:

- Already have finalized distances
- Must not be updated again

◆ PROMPT 16: What is relaxation in this code?

```
if d[u] + wgt < d[v]:
```

✅ Answer:

This checks whether:

Going from `src → u → v` is shorter than the current best path to `v`

◆ PROMPT 17: What happens if relaxation succeeds?

```
d[v] = d[u] + wgt

pq.update(pqlocator[v], d[v], v)
```

✅ Answer:

- Distance estimate is improved
- Priority queue key is updated
- Vertex `v` may rise in priority

◆ PROMPT 18: Why is `pq.update()` critical?

✅ Answer:

Without it:

- Priority queue would have outdated distances
- Algorithm would break
- Time complexity would worsen

◆ PROMPT 19: Why are unreachable vertices excluded?

```
return cloud
```

✅ Answer:

Vertices with ∞ distance:

- Were never relaxed
- Are not reachable from `src`
- Are intentionally omitted

◆ PROMPT 20: Simple execution visualization

🖊 Graph

```
s --2--> a --1--> b
 \--5--> b
```

🖊 Steps

```
Start:

D[s]=0, D[a]=∞, D[b]=∞


Pick s:

Relax a → D[a]=2

Relax b → D[b]=5


Pick a:

Relax b → D[b]=3


Pick b:

Done
```

✅ Final cloud

```
{s:0, a:2, b:3}
```

◆ PROMPT 21: Time complexity of this implementation

✅ Answer:

Using an adaptable heap:

$$O((V+E)\log V$$

◆ PROMPT 22: When does this implementation fail?

❌ Fails if:

- Graph has **negative edge weights**

✅ Use instead:

- **Bellman–Ford algorithm**

◆ PROMPT 23: Key differences from BFS

BFS	Dijkstra
Unweighted	Weighted
Queue	Priority Queue
O(V+E)	O((V+E)logV)

◆ PROMPT 24: One-picture mental model

```
Cloud grows outward

closest vertex first

distance never decreases

   s

  / \

 a   c

 |

 b
```